

# Design the Data Model

## 1. Overview

After clarifying requirements and drafting APIs, next step is:

Design how data will be structured, stored, queried, and evolved.

This determines:

- Performance
- Scalability
- Query efficiency
- Maintainability

## 2. Why Data Model is Critical

Bad schema causes:

- Slow queries
- Difficult scaling
- Complex migrations
- Tight coupling
- Future redesign cost

## 3. First Question to Ask

Before designing:

**Are we:**

## A. Building New System?

Design schema from scratch.

## B. Integrating Existing Systems?

Need:

- APIs
- ETL pipelines
- Compatibility planning

# 4. Shared Database Across Services (Anti-Pattern)

Very common interview topic.

## ✗ What is Shared DB?

Multiple services directly use same DB tables.

None

Order Service → Same DB ← User Service

Inventory Service → Same DB

# Why It's Bad

## 1. Resource Contention

Heavy query by one service slows others.

Example:

- Large UPDATE locks rows

## 2. Coupled Schema Changes

Changing one table may break another service.

## 3. Team Dependency

Teams must coordinate every DB change.

Leads to:

- Meetings
- Delays
- Friction

## 4. Technology Lock-in

All services forced to use same DB.

Maybe:

- Feed needs Cassandra
- Billing needs PostgreSQL
- Search needs Elasticsearch

But shared DB blocks this.

## 5. Better Pattern: Database per Service

None

User Service → User DB

Order Service → Order DB

Feed Service → Feed DB

### Benefits:

- Independent scaling
- Independent schema changes
- Polyglot persistence
- Team autonomy

## 6. Polyglot Persistence

Use best DB for each use case.

| Use Case      | Good Choice |
|---------------|-------------|
| Transactions  | PostgreSQL  |
| Key-value     | Redis       |
| Feed Timeline | Cassandra   |

|        |               |
|--------|---------------|
| Search | Elasticsearch |
| Graph  | Neo4j         |

Mentioning this = maturity.

## 7. How to Design Schema

Use APIs as input.

Example APIs:

None

**POST** /posts

**GET** /posts/{id}

Schema:

SQL

Posts(

post\_id,

user\_id,

content,

created\_at

)

## 8. Map APIs to Tables

### Write APIs

POST / PUT → Insert / Update schema

### Read APIs

GET → Query optimized indexes

## 9. Example: Social App Schema

### Users Table

SQL

```
Users(  
  id PK,  
  name,  
  email,  
  created_at  
)
```

## Posts Table

SQL

```
Posts(  
  id PK,  
  user_id FK,  
  text,  
  created_at  
)
```

## Followers Table

SQL

```
Followers(  
  follower_id,  
  followee_id  
)
```

# 10. Query-Driven Design

Design schema based on access patterns.

Ask:

- What reads are frequent?
- What writes are frequent?
- Need sorting?
- Need joins?

Example:

If feed reads dominate:

Denormalize feed tables.

# 11. Normalization vs Denormalization

## Normalized

Separate tables.

Pros:

- Clean data
- Less duplication

Cons:

- More joins

## Denormalized

Duplicate data for speed.

Pros:

- Faster reads

Cons:

- Update complexity

Use depending on workload.

## **12. Scale Considerations**

Ask:

- Rows per day?
- Storage/year?
- Hot partitions?
- Need sharding?

## **13. Security Considerations**

- PII encryption
- Access controls
- Audit logs
- Row-level permissions

## **14. Schema Evolution**

Need support for:

- Backward compatibility
- Additive changes first
- Dual writes during migration

## 15. Data Pipelines (ETL)

If integrating systems:

None

OLTP DB → Kafka → Warehouse

Used for:

- Analytics
- Reporting
- ML

## 16. Interview Script

Say:

“I’d prefer database-per-service to avoid tight coupling. I’ll derive the schema from core APIs and optimize based on access patterns. For analytics or cross-service needs, I’d expose APIs or ETL pipelines rather than direct DB sharing.”

That sounds strong.

## 17. Example: E-commerce

None

User Service → Users DB

Catalog Service → Product DB

Order Service → Orders DB

Payment Service → Ledger DB

Search Service → Search Index

## 18. Final Summary

- Data model drives scalability & performance
- Prefer DB-per-service over shared DB
- Derive schema from APIs & access patterns
- Choose DB type per use case
- Plan for migrations & analytics

## What Interviewers Love Here

When you say:

“Schema should follow query patterns.”

That’s senior-level thinking.